

Minimizing Language With Emojis

Jacob Lorenzo¹, Johnny Sylvain²

{jacob.w.lorenzo¹, john.sylvain²}@umaine.edu
University of Maine

May 1, 2026

Abstract

We present a system that inserts semantically appropriate emojis into natural language sentences. Our system uses a frozen pretrained sentence encoder and encodes each emoji to a 768-dimensional embedding space. There are no learned parameters, each embedding is a deterministic average over a sentence, phrase, or word related to the emoji. We infer the best emoji to use with a dot-product lookup and a greedy non-overlapping selection.

The baseline averages the embeddings of every full sentence which an emoji appears in, from a merged 504k row corpus (KomeijiForce/Text2Emoji and MLap/Sentiment2Emoji). The second method uses the embeddings of each emoji’s Unicode short name without the corpus. The third method chunks the text into n-gram phrases looking for the closest embedding of the Unicode short name with neighbor smoothing.

Qualitative evaluation on 5 test sentences shows that method 2 (names-only) produces the clearest and most precise outputs as the mapping between common nouns and Unicode short names is often matched 1:1. The phrase aligned method (3) expands the versatility of names-only method (2), but inherits some bias and confidence misclassifications from full sentence method (1). We discuss the dataset and methodological factors driving each pattern, and identify longer n-gram budgets and more diverse data as the most promising next steps.

1 Dataset

This project uses two datasets hosted publicly on Hugging Face.

Sentiment2Emoji (Table 1), published in 2025 by Aman Prakash, is designed for sentiment analysis and emotion classification tasks. This dataset is repurposed as a supplement to our main corpus. The dataset has sentences and strictly 3 emojis that describe the sentence.

Number of combined rows: 357 | Total file size: 19.7 kB

Table 1: Sample rows in the Sentiment2Emoji Dataset

Sentence	Emoji
I just received an unexpected package in the mail	  
The waves were perfect for surfing today	  
My soufflé collapsed in the oven	  

Text2Emoji (Table 2), published on 3 Nov 2023, alongside the paper EmojiLM: Modeling the New Emoji Language by Letian Peng et al. is our main corpus. It was created by synthesizing data

from LLMs. The dataset has sentences and a cluster of 2-10 emojis related to the sentence. The Text2Emoji dataset also has an attached topic string for each row, but we do not use it. Number of rows: 503,687 | Total file size: 66.9 MB

Table 2: Sample rows in the Text2Emoji Dataset

text	emoji	topic
Being a nurse is a rollercoaster of emotions, from comforting patients to dealing with medical emergencies.	         	career
Can't wait to finally see my best friend tomorrow, I have missed them so much!	    	feeling
Pure bliss! Spend an entire day doing what you love can light up your soul like nothing else	         	feeling

The third dataset we worked with later is emoji_data_python module (Table 3). This module has 489 entries. This dataset is used separately from the others, using the English short names for the “text” column of a dataframe. :woman_health_worker: -> “woman_health_worker”

Table 3: Example text emoji dataframe from the emoji_data python module

text	emoji
“woman_health_worker”	

1.1 Preprocessing

The first step is to convert our data into pandas dataframes. The dataframes have two renamed columns; text and emoji. We concatenate the dataframes row wise to create a 504k row corpus. This is saved as a CSV in z_data/merged.csv for inspection.

Next, we drop any row where there is an empty text or emoji cell. Then we create a new column “emoji_list” that splits the emoji column into a python list rather than a string for each row so each emoji is protected. This is critical as some emojis are combinations of Unicode, so each index is cleanly one emoji. We also clean the sentences to remove stop words, whitespace, and punctuation using spaCy, additionally disabling the NER and parser components for speed. This strengthens the connection between emoji and the most important parts of the sentences.

We then vectorized using sentence-transformers/all-mpnet-base-v2 which maps sentences & paragraphs to a 768 dimensional dense vector space.

To vectorize the emoji we look at each unique emoji, for each row (text translation) the emoji appears, and average the vectors into a 768 dimensional dense vector space. This set of vectors and emojis is stored as vectors.npy and emojis.pkl to avoid rerunning this expensive step.

1.2 Train/validation/test split

We do not have a Train/validation/test split. We’re using a pre-existing model and getting vectors from it via encoding. We use those encodings as the basis for our cosine similarity. A train/test split would catch overfitting, but nothing is being fit, so there is nothing to overfit. We instead evaluate output quality qualitatively: Is the meaning of the test sentence maintained?

2 Methodology

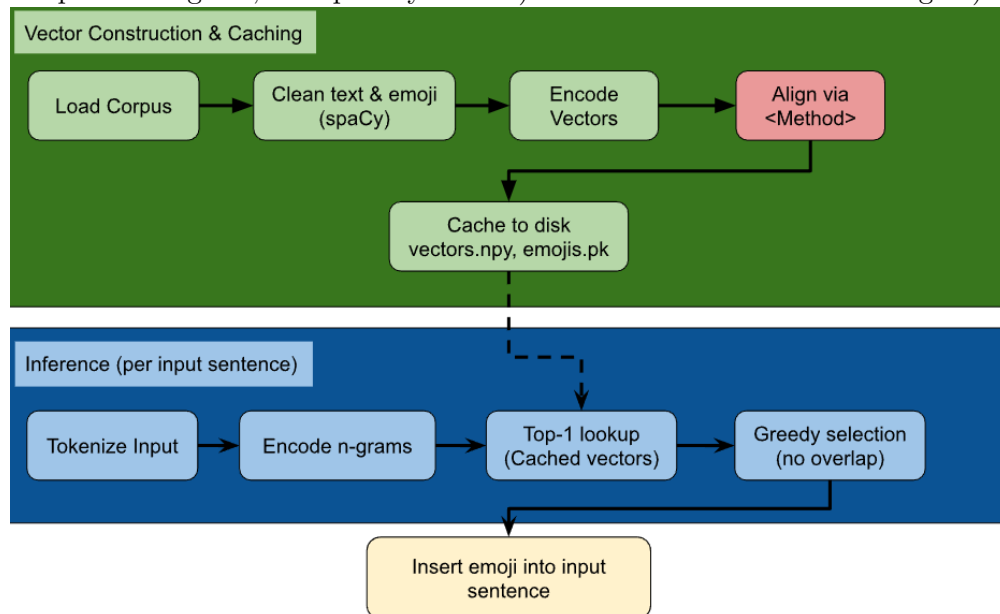
This section describes the system’s pipeline, the baseline approach, parameters, tools and libraries used. We use a frozen pretrained encoder and produce vectors for each emoji by deterministic averaging. The unit of text contributing to the emojis is different for our baseline and methods.

1. Method 1 - The baseline is whole sentence aggregation of the vectors, producing centroids per emoji.
2. Method 2 - Without the corpus, a direct embedding of the Unicode english name mapped directly to each emoji.
3. Method 3 - Tak chunks of sentences rather than the entire sentences from the corpus. Generating phrase-aligned average that assigns each emoji to the n-gram chunk of the sentence whose embedding is closest to the emoji’s Unicode short name. This method also supports smoothing the best chunk vs. neighbors.

2.1 Pipeline

The pipeline is consistent for each alignment method. There are two distinct states for our pipeline: 1) Vector construction & Caching 2) Inference. The first part loads the data into a coherent corpus, cleans those data, and encodes the vectors. This process is described in Figure 1.

Figure 1: Pipeline Diagram, Two part system. 1) Vector construction & Caching 2) Inference



2.2 Components

2.2.1 tokenizer.py

```
process_emojis(emoji_str: str) -> list[str]
```

Takes a string of emojis and converts it to a list of emojis.

clean_spacy_single(text: str, nlp: Language) -> list[Token]

Takes a sentence, strips the punctuation, spaces, and stop words, then returns the remaining tokens.

clean_spacy_batch(text: list[str], nlp: Language, batch: int, args: Namespace) -> list[str]

Takes a list of sentences and processes them in a batch by removing the punctuation, spaces, and stop words. Returns a list of strings of the cleaned sentences.

extract_ngram(tokens: list[Token], n: int) -> list[tuple[str, int, int]]

Takes a single list of tokens (or one sentence) and converts it to a list of n-grams that hold their indices.

get_emoji_name(e: str) -> str | None

Takes an emoji name from the emoji library dictionary and processes it by removing underscores and colons.

2.2.2 load_data.py

make_data_dir() -> None

Makes the directory used for storing the relevant datasets and vectors.

data_exists() -> bool

Checks if vectors.npy and emojis.pkl exist.

load_or_download(path: Path, repo_id: str, filename: str) -> Path

Downloads a given dataset from the hub if the path to the file doesn't exist.

get_dataset_contents() -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]

Retrieves the necessary datasets from either local or from the hub. Returns 3 data frames for each of the respective datasets.

process_dataframes(df_1: pd.DataFrame, df_2: pd.DataFrame, df_3: pd.DataFrame) -> pd.DataFrame

Normalizes the column names and merges the datasets.

save_data(vec_list: NDArray[np.float32], emojis: list[str]) -> None

Saves the numpy array to the local directory and pickles the list of emojis.

load_data() -> tuple[NDArray[np.float32], list[str]]

Loads the numpy array and the pickled emojis.

2.2.3 main.py

create_emoji_mapping(merged: pd.DataFrame, model: SentenceTransformer, nlp: Language, args: Namespace) -> list[tuple[str, NumpyArray[np.float32]]]

Creates a list of tuples that contain the emoji alongside a vector that is an average of all the encoded vectors that contained the emoji in their translation.

create_emoji_mapping_phrase(merged: pd.DataFrame, model: SentenceTransformer, nlp: Language, args: Namespace) -> list[tuple[str, NumpyArray[np.float32]]]

Split the sentence into n-gram chunks and assign each emoji to whichever chunk best matches its unicode short name. Each emoji's centroid is built from chunk vectors smoothed with their immediate neighbors.

encoder(model: SentenceTransformer, t_input: list[str] | str | pd.Series, args: Namespace) -> NumpyArray[np.float32]

Uses a sentence transformer to batch encode sentences. Returns an array of vectors.

initialize_data(model: SentenceTransformer, nlp: Language, args: Namespace) -> tuple[list[str], NumpyArray[np.float32]]

Processes the data in the data frames through generation of sentence mappings. Returns a list of emojis and a list of their associated vectors.

get_similarities(vector: NumpyArray[np.float32], vectors: NumpyArray[np.float32]) -> NumpyArray[np.float32]

Performs a cosine similarity between the n-gram vector and the array of emoji vectors.

get_top_k(similarities: NumpyArray[np.float32], k: int) -> NumpyArray[np.intp]

Returns the indices of the top k similar vectors.

overlap(s1: int, e1: int, s2: int, e2: int) -> bool

Checks if an n-gram would overlap with an existing n-gram

duplicate(em1: str, em2: str) -> bool

Checks if an emoji would be a duplicate in the translation.

get_emoji_slices(selection_sorted: list[tuple[tuple[str, int, int], str, np.float32]], args: Namespace) -> list[tuple[tuple[str, int, int], str, np.float32]]

Determines which emojis from the list of candidates are suitable for inclusion based on if it would overlap with a preexisting n-gram or if it's a duplicate.

construct_sentence(selected_sorted: list[tuple[tuple[str, int, int], str, np.float32]], word_input: str) -> str

Iterates over the selected emoji slices and uses their beginning and end indices for slicing into the text.

run_translation(model: SentenceTransformer, nlp: Language, vec_array: NDArray[np.float32], emojis: list[str], args: Namespace) -> None

Takes a given input sentence, slices it into n-grams, gets the similarity vectors for each n-gram, then gets the top k emojis based on the highest similarities. Afterward, the list is sorted by their similarity and slices are determined based on the best fit.

check_positive(value: str)

Used for argparser to determine if a user argument is a positive, nonzero integer.

build_parser() -> Namespace

Constructs the argument parser.

2.3 Hyper-parameters & Tools

Our system uses flags to change between methods and modify the n-gram bounds, neighbor smoothing, emoji replacements, encoder batch size. The system uses 5 libraries.

Table 4: Hyperparameters, flags and defaults

Hyperparameter	Flag	Default	Effect
Max n-gram size	-ngram	2	Upper bound on chunk length, both for centroid construction (phrase mode) and inference.
Smoothing weight	-smooth	0.7	Phrase mode only. Mass on the best chunk vs. neighbor mean.
Replacements R	-repl	4	Maximum non-overlapping emoji insertions per output sentence.
Encoder batch size	-batch	64	Larger values are faster on GPU but use more memory.
Alignment method	-phrase / -emoji	whole-sentence	Selects baseline, phrase-aligned, or names-only.

Table 5: Libraries used

Library	Use
sentence-transformers	Encoder
spaCy with en_core_web_sm	tokenization and stop-word filtering
pandas	Corpus merge and data storage
emoji	grapheme splitting and Unicode-name lookup
numpy	similarity search

3 Results

Using the default n-gram size (2) and number of n-grams to replace in input sentence (4) we sampled 5 input sentences using each method. Tables 6, 7, and 8 show the input sentence, the encoded words to emoji and the output sentence.

The emojis shown in this paper are typeset via the `\texttt{emoji}` package under LuaLaTeX, which maps each emoji to a glyph drawn from a single emoji font (Noto Color Emoji on our build). Some emojis produced by our system involve zero-width-joiner sequences, skin-tone modifiers, or multi-tone combinations (for example, two people holding hands with different skin tones). Coverage of these compound glyphs varies between emoji fonts and Unicode versions.

Table 6: Method 1 - Baseline full sentence averaging

#	Input Sentence	Encoding	Output sentence
1	Rain or shine we play outside all day long	[('Rain', 0, 4),  , 0.616), (('shine', 8, 13),  , 0.426), (('play outside', 17, 29),  , 0.523), (('day long', 34, 42),  , 0.317)]	 or  we  all 
2	The quick brown fox jumps over the lazy dog	[('brown fox', 10, 19),  , 0.474), (('jumps', 20, 25),  , 0.403), (('lazy', 35, 39),  , 0.291), (('dog', 40, 43),  , 0.364)]	The quick   over the 
3	I went on a grand adventure, where I found buried treasure and married the love of my life	[('adventure', 18, 27),  , 0.593), (('buried treasure', 43, 58),  , 0.420), (('love', 75, 79),  , 0.467), (('life', 86, 90),  , 0.437)]	I went on a grand  , where I found  and married the  of my 
4	A student drank coffee before the test	[('student', 2, 9),  , 0.333), (('drank', 10, 15),  , 0.309), (('coffee', 16, 22),  , 0.578), (('test', 34, 38),  , 0.195)]	A    before the 
5	I need help with this coding problem, what is the bug?	[('need', 2, 6),  , 0.311), (('help', 7, 11),  , 0.308), (('coding', 22, 28),  , 0.572), (('problem', 29, 36),  , 0.314)]	I   with this   , what is the bug?

Method 1 (sentences) has a clear bias toward person emojis. Across all 20 selections it picks people emojis (, , , , , , , ...) for concepts that are not people: *rain*, *shine*, *lazy*, *life*, *problem*.

Method 2 (names-only) is the cleanest of the three. In sentences 2, 4, and 5 it returns / for *fox/dog*, // for *student/coffee/test*, and  for *bug*. Several of these reach a confidence of 1.0 because the n-gram is literally identical to the emoji’s Unicode short name.

Method 3 (phrases) is mixed. Real wins:  for *rain* in sentence 1, the *outside day* 2-gram resolving to  (correctly capturing the “outdoor activity” meaning), and  for *bug*. Real losses:  (the Armenian flag) for *adventure*,  for *life*, and  for *brown*. Phrase alignment helped on some emojis and clearly did not on others.

Table 7: Method 2 - Embed of the unicode english name to each emoji

#	Input Sentence	Encoding	Output sentence
1	Rain or shine we play outside all day long	[('Rain', 0, 4), 🌧️, 0.668), (('shine', 8, 13), 🌟, 0.609), (('play', 17, 21), 🎮, 0.603), (('outside', 22, 29), 🌿, 0.550)]	🌧️ or 🌟 we 🎮 🌿 all day long
2	The quick brown fox jumps over the lazy dog	[('brown', 10, 15), 🟤, 0.666), (('fox', 16, 19), 🦊, 1.000), (('jumps', 20, 25), 🏊, 0.508), (('dog', 40, 43), 🐶, 1.000)]	The quick 🟤 🦊 🏊 over the lazy 🐶
3	I went on a grand adventure, where I found buried treasure and married the love of my life	[('adventure', 18, 27), 🎮, 0.560), (('buried', 43, 49), 🍷, 0.653), (('treasure', 50, 58), 💰, 0.515), (('love', 75, 79), 💕, 0.567)]	I went on a grand 🎮, where I found 🍷 💰 and married the 💕 of my life
4	A student drank coffee before the test	[('student', 2, 9), 🧑, 1.000), (('coffee', 16, 22), ☕, 0.553), (('test', 34, 38), 📝, 0.594)]	A 🧑 drank ☕ before the 📝
5	I need help with this coding problem, what is the bug?	[('need', 2, 6), 📦, 0.414), (('help', 7, 11), ⚠️, 0.364), (('coding', 22, 28), 🤖, 0.500), (('bug', 50, 53), 🐛, 1.000)]	I 📦 ⚠️ with this 🤖 problem, what is the 🐛?

Table 8: Method 3 - Sentence chunking results

#	Input Sentence	Encoding	Output sentence
1	Rain or shine we play outside all day long	[('Rain', 0, 4), 🌧️, 0.700), (('shine', 8, 13), 🧑, 0.487), (('play', 17, 21), 🏊, 0.593), (('outside day', 22, 37), 🌿, 0.670)]	🌧️ or 🧑 we 🏊 🌿 long
2	The quick brown fox jumps over the lazy dog	[('brown', 10, 15), 🏊, 0.567), (('fox', 16, 19), 🦊, 0.548), (('jumps', 20, 25), 🏊, 0.458), (('dog', 40, 43), 🐶, 0.588)]	The quick 🏊 🦊 🏊 over the lazy 🐶
3	I went on a grand adventure, where I found buried treasure and married the love of my life	[('adventure', 18, 27), 🇷🇺, 0.818), (('found', 37, 42), ✨, 0.631), (('love', 75, 79), 🧑, 0.897), (('life', 86, 90), 🧑, 0.487)]	I went on a grand 🇷🇺, where I ✨ buried treasure and married the 🧑 of my 🧑
4	A student drank coffee before the test	[('student', 2, 9), 🧑, 0.537), (('drank', 10, 15), 📝, 0.390), (('coffee', 16, 22), ☕, 0.599), (('test', 34, 38), 🧑, 0.326)]	A 🧑 ☕ before the 🧑
5	I need help with this coding problem, what is the bug?	[('help', 7, 11), 🧑, 0.848), (('coding', 22, 28), 🤖, 0.631), (('bug', 50, 53), 🐛, 0.332)]	I need 🧑 with this 🤖 problem, what is the 🐛?

4 Discussion

We believe that while using method 2 will accurately represent any part of common words in a sentence being replaced, it is limited to the existing 489 emoji. If a sentence is full of words that are outside of the context of the 489 literal emoji translations, the selected emoji will be at the same level as Method 3.

Method 3 finds a middle ground between the baseline method and method 2, allowing for more versatile emoji use at the expense of the precision of a more literal model.

The issue with the misalignment of person emojis in methods 1 and 3 are a twofold issue. The dataset has “\u200d” often to reflect a gender swapping for all emoji, not just person emojis. Additionally, the dataset has multi-emoji clusters where the connection between any single emoji and any single phrase is loose.

4.1 Improvements

The n-gram budget at inference (n-gram 2 by default). Concepts that span longer phrases (“roller-coaster of emotions” and “love of my life”) are visible to the system only as their individual tokens or 2-grams; the relevant 3-grams and 4-grams are never scored. Raising the n-gram budget linearly increases inference cost and construction cost in phrase mode, but might be worth it for outputs that contain idiomatic phrases.

Improving the diversity of the data, so each emoji has a similar count over the entire corpus would likely improve misclassifications. Underrepresented emojis may take on additional or incorrect meaning and overrepresented emojis may never be the top choice as they are stretched.

References

- [1] L. H. Klein, R. Aydin, and R. West, “Emojinize: Enriching Any Text with Emoji Translations,” arXiv.org, 2024. <https://arxiv.org/abs/2403.03857> (accessed Mar. 13, 2026).
- [2] C. Huang, S. Xie, and B. Zhang, “Emojify: Emoji Prediction from Sentence.” Available: https://cs229.stanford.edu/proj2019aut/data/assignment_308832_raw/26647690.pdf
- [3] W. Yuqiao, W. Ye, M. Zhao, and R. Wattenhofer, “Emoji for Natural Language Processing,” 2021. Available: <https://pub.tik.ee.ethz.ch/students/2020-HS/MA-2020-31.pdf>
- [4] aman prakash, “Sentiment2Emoji,” Huggingface.co, 2025. <https://huggingface.co/datasets/MLap/Sentiment2Emoji> (accessed Feb. 12, 2026).
- [5] S. H. Cappallo, “Twemoji Dataset”. University of Amsterdam / Amsterdam University of Applied Sciences, 25-Jan-2018 [Online]. Available: https://uva.uas.figshare.com/articles/dataset/Twemoji_Dataset/5822100/3. [Accessed: 12-Feb-2026]
- [6] L. Peng, “Text2Emoji,” Huggingface.co, Nov. 10, 2023. <https://huggingface.co/datasets/KomeijiForce/Text2Emoji> (accessed Mar. 13, 2026).
- [7] L. Peng, Z. Wang, H. Liu, Z. Wang, and J. Shang, “EmojiLM: Modeling the New Emoji Language,” arXiv.org, 2023. <https://arxiv.org/abs/2311.01751>

Appendix

Source code: JWLorenzo/COS_498_598_Project_GROUP_3